

Lecture #8 Advanced Priority Queues

Binomial, Fibonacci and Rank-Pairing Heaps

Rodica Potolea, Camelia Lemnaru, Ciprian Oprea and Raluca
Laura Portase

Technical University of Cluj-Napoca
Computer Science



Agenda

- 1 Motivation & Recap
- 2 Binomial Heaps
- 3 Fibonacci Heaps
- 4 Rank-Pairing Heaps



Priority Queues Recap

- We maintain a dynamic set of elements with *keys*



Priority Queues Recap

- We maintain a dynamic set of elements with *keys*
- Focus on efficiently finding/removing the highest/lowest priority element



Priority Queues Recap

- We maintain a dynamic set of elements with *keys*
- Focus on efficiently finding/removing the highest/lowest priority element
- Core operations:



Priority Queues Recap

- We maintain a dynamic set of elements with *keys*
- Focus on efficiently finding/removing the highest/lowest priority element
- Core operations:
 - **Insert(x)** – add element x to the set



Priority Queues Recap

- We maintain a dynamic set of elements with *keys*
- Focus on efficiently finding/removing the highest/lowest priority element
- Core operations:
 - **Insert(x)** – add element x to the set
 - **FindMin()** – return element with minimum key



Priority Queues Recap

- We maintain a dynamic set of elements with *keys*
- Focus on efficiently finding/removing the highest/lowest priority element
- Core operations:
 - **Insert(x)** – add element x to the set
 - **FindMin()** – return element with minimum key
 - **ExtractMin()** – remove and return the minimum



Priority Queues Recap

- We maintain a dynamic set of elements with *keys*
- Focus on efficiently finding/removing the highest/lowest priority element
- Core operations:
 - **Insert(x)** – add element x to the set
 - **FindMin()** – return element with minimum key
 - **ExtractMin()** – remove and return the minimum
 - **DecreaseKey(x , newKey)** – decrease the key of x



Priority Queues Recap

- We maintain a dynamic set of elements with *keys*
- Focus on efficiently finding/removing the highest/lowest priority element
- Core operations:
 - **Insert(x)** – add element x to the set
 - **FindMin()** – return element with minimum key
 - **ExtractMin()** – remove and return the minimum
 - **DecreaseKey(x , newKey)** – decrease the key of x
 - (Sometimes: **Delete(x)**, **Merge($H1$, $H2$)**)



Priority Queues Recap

- We maintain a dynamic set of elements with *keys*
- Focus on efficiently finding/removing the highest/lowest priority element
- Core operations:
 - **Insert(x)** – add element x to the set
 - **FindMin()** – return element with minimum key
 - **ExtractMin()** – remove and return the minimum
 - **DecreaseKey(x , newKey)** – decrease the key of x
 - (Sometimes: **Delete(x)**, **Merge($H1$, $H2$)**)
- Implementation so far: **binary heap**



Priority Queues Recap

- We maintain a dynamic set of elements with *keys*
- Focus on efficiently finding/removing the highest/lowest priority element
- Core operations:
 - **Insert(x)** – add element x to the set
 - **FindMin()** – return element with minimum key
 - **ExtractMin()** – remove and return the minimum
 - **DecreaseKey(x , newKey)** – decrease the key of x
 - (Sometimes: **Delete(x)**, **Merge($H1$, $H2$)**)
- Implementation so far: **binary heap**
- Typical time bounds for n elements:



Priority Queues Recap

- We maintain a dynamic set of elements with *keys*
- Focus on efficiently finding/removing the highest/lowest priority element
- Core operations:
 - **Insert(x)** – add element x to the set
 - **FindMin()** – return element with minimum key
 - **ExtractMin()** – remove and return the minimum
 - **DecreaseKey(x , newKey)** – decrease the key of x
 - (Sometimes: **Delete(x)**, **Merge($H1$, $H2$)**)
- Implementation so far: **binary heap**
- Typical time bounds for n elements:
 - Insert, ExtractMin, DecreaseKey: all $\mathcal{O}(\log n)$



Priority Queues Recap

- We maintain a dynamic set of elements with *keys*
- Focus on efficiently finding/removing the highest/lowest priority element
- Core operations:
 - **Insert(x)** – add element x to the set
 - **FindMin()** – return element with minimum key
 - **ExtractMin()** – remove and return the minimum
 - **DecreaseKey(x , newKey)** – decrease the key of x
 - (Sometimes: **Delete(x)**, **Merge($H1$, $H2$)**)
- Implementation so far: **binary heap**
- Typical time bounds for n elements:
 - Insert, ExtractMin, DecreaseKey: all $\mathcal{O}(\log n)$
 - Merge: $\mathcal{O}(n)$ (by rebuild)



Binary Heap – Recap

- Complete binary tree with partial ordering relation (represented as array)
- We assume a **min-heap**: key at parent $\leq$ keys of children
- **FindMin()**



Binary Heap – Recap

- Complete binary tree with partial ordering relation (represented as array)
- We assume a **min-heap**: key at parent $\leq$ keys of children
- **FindMin()**
 - Minimum = element at the root (index 1 in the array)
 - Time: $\mathcal{O}(1)$



Binary Heap – Recap

- Complete binary tree with partial ordering relation (represented as array)
- We assume a **min-heap**: key at parent $\leq$ keys of children
- **FindMin()**
- **Insert(x)**



Binary Heap – Recap

- Complete binary tree with partial ordering relation (represented as array)
- We assume a **min-heap**: key at parent $\leq$ keys of children
- **FindMin()**
- **Insert(x)**
 - Add x at the end of the array (next free position)
 - *Bubble up* while $\text{key}[x] < \text{key}[\text{parent}(x)]$
 - Time: $\mathcal{O}(\log n)$



Binary Heap – Recap

- Complete binary tree with partial ordering relation (represented as array)
- We assume a **min-heap**: key at parent $\leq$ keys of children
- **FindMin()**
- **Insert(x)**
- **ExtractMin()**



Binary Heap – Recap

- Complete binary tree with partial ordering relation (represented as array)
- We assume a **min-heap**: key at parent $\leq$ keys of children
- **FindMin()**
- **Insert(x)**
- **ExtractMin()**
 - Save min at root, move last element to the root
 - *Bubble down* (heapify) to restore heap property
 - Time: $\mathcal{O}(\log n)$



Binary Heap – Recap

- Complete binary tree with partial ordering relation (represented as array)
- We assume a **min-heap**: key at parent $\leq$ keys of children
- **FindMin()**
- **Insert(x)**
- **ExtractMin()**
- **DecreaseKey(x, newKey)**



Binary Heap – Recap

- Complete binary tree with partial ordering relation (represented as array)
- We assume a **min-heap**: key at parent $\leq$ keys of children
- **FindMin()**
- **Insert(x)**
- **ExtractMin()**
- **DecreaseKey(x, newKey)**
 - Decrease key of node x to *newKey*
 - *Bubble up* as long as heap property is violated
 - Time: $\mathcal{O}(\log n)$



Binary Heap – Recap

- Complete binary tree with partial ordering relation (represented as array)
- We assume a **min-heap**: key at parent $\leq$ keys of children
- **FindMin()**
- **Insert(x)**
- **ExtractMin()**
- **DecreaseKey(x, newKey)**
- **Delete(x)** (optional)



Binary Heap – Recap

- Complete binary tree with partial ordering relation (represented as array)
- We assume a **min-heap**: key at parent $\leq$ keys of children
- **FindMin()**
- **Insert(x)**
- **ExtractMin()**
- **DecreaseKey(x, newKey)**
- **Delete(x)** (optional)
 - Typical trick: $\text{DecreaseKey}(x, -\infty) + \text{ExtractMin}()$
 - Time: $\mathcal{O}(\log n)$



Why More Advanced Heaps?

- Many graph algorithms use priority queues **heavily**:
 - Dijkstra, Prim, A*, etc
- Example: Dijkstra
 - One **ExtractMin** per vertex $\Rightarrow \mathcal{O}(V \log V)$
 - Up to one **DecreaseKey** per edge $\Rightarrow \mathcal{O}(E \log V)$
- Question:
 - Can we make **DecreaseKey** and **Insert** almost “free”?
 - i.e. $\mathcal{O}(1)$ *amortized* time
- Answer:
 - Yes, but we must go beyond binary heaps:
 - **Binomial Heaps** (structured forest)
 - **Fibonacci Heaps** (lazy binomial-like structure)
 - **Rank-Pairing Heaps** (simplified Fibonacci-style heap)



Binomial Trees B_k – Properties

- **Degree-based augmented trees**, denoted B_k
- **Degree of a binomial tree** = number of children of the root:
In a B_k , the root has degree k
- **Properties of a B_k tree:**
 - **P1:** Degree of root: $\text{Degree}(\text{root}[B_k]) = k$
 - **P2:** Number of nodes in B_k : $|B_k| = 2^k$
 - **P3:** $\text{Height}(B_k) = k$
 - **P4:** Nb. of nodes at level i : #nodes at level $i = \binom{k}{i}$
 - **P5:** All children of the root, numbered left-to-right as $k-1, k-2, \dots, 1, 0$ – satisfy: **child i is a B_i tree**
- **Implication:**
 - **P5** provides one recursive definition of B_k



Binomial Trees B_k – Recursive Definitions

- Two equivalent recursive views:
- **Definition 1 (2 copies):**
 - A B_k tree is formed by taking **two** B_{k-1} **trees** and linking their roots (one root becomes the leftmost child of the other)
- **Definition 2 (children view):**
 - A B_k tree consists of k children that are themselves:

$$B_{k-1}, B_{k-2}, \dots, B_1, B_0$$

(ordered left-to-right by decreasing size, as in **P5**)

- **These two definitions are equivalent:**
 - Definition 1 implies children: B_{k-1} and B_{k-1} , and recursively these form $B_{k-2}, B_{k-3}, \dots$
 - Definition 2 collapses upward to the “two B_{k-1} trees” view



Proof Sketch of P4: #nodes at level $i = \binom{k}{i}$

- We prove by induction on k that level i of B_k has $\binom{k}{i}$ nodes
- Consider the recursive definition:

$$B_k = \text{two linked } B_{k-1}\text{s}$$



Proof Sketch of P4: #nodes at level $i = \binom{k}{i}$

- We prove by induction on k that level i of B_k has $\binom{k}{i}$ nodes
- Consider the recursive definition:

$$B_k = \text{two linked } B_{k-1}\text{'s}$$

- On level i of B_k we have:



Proof Sketch of P4: #nodes at level $i = \binom{k}{i}$

- We prove by induction on k that level i of B_k has $\binom{k}{i}$ nodes
- Consider the recursive definition:

$$B_k = \text{two linked } B_{k-1}\text{'s}$$

- On level i of B_k we have:
 - From the first B_{k-1} (the one whose root becomes the root of B_k):

$$\binom{k-1}{i} \text{ nodes}$$



Proof Sketch of P4: #nodes at level $i = \binom{k}{i}$

- We prove by induction on k that level i of B_k has $\binom{k}{i}$ nodes
- Consider the recursive definition:

$$B_k = \text{two linked } B_{k-1}\text{'s}$$

- On level i of B_k we have:
 - From the first B_{k-1} (the one whose root becomes the root of B_k):

$$\binom{k-1}{i} \text{ nodes}$$

- From the second B_{k-1} :

$$\binom{k-1}{i-1} \text{ nodes}$$

(one level deeper, since its root becomes a child)



Proof Sketch of P4: #nodes at level $i = \binom{k}{i}$

- We prove by induction on k that level i of B_k has $\binom{k}{i}$ nodes
- Consider the recursive definition:

$$B_k = \text{two linked } B_{k-1}\text{s}$$

- Therefore, total nodes at level i :

$$\binom{k-1}{i} + \binom{k-1}{i-1} = \binom{k}{i}$$



Proof Sketch of P4: #nodes at level $i = \binom{k}{i}$

- We prove by induction on k that level i of B_k has $\binom{k}{i}$ nodes
- Consider the recursive definition:

$$B_k = \text{two linked } B_{k-1}\text{'s}$$

- Therefore, total nodes at level i :

$$\binom{k-1}{i} + \binom{k-1}{i-1} = \binom{k}{i}$$

- **Conclusion:** Levels of B_k follow the binomial coefficients



Binomial Heaps – Definition and Structure

Definition. A *binomial heap* H is a collection of binomial trees satisfying:

- **(P1)** Each node has a key (each tree is min-heap ordered)
- **(P2)** For every degree k , H contains **at most one** tree B_k
- **(P3)** All trees are stored in a root list, sorted by increasing deg.

How many trees can H have?

Let the degrees of the t trees be: $d_0 < d_1 < \dots < d_t$

Then: $n = \sum_{i=0}^t 2^{d_i}$

Since the degrees are distinct, we have $d_i \geq i$

Thus: $n \geq \sum_{i=0}^t 2^i = 2^{t+1} - 1$

Therefore:

$$t + 1 \leq \lfloor \lg(n + 1) \rfloor \leq \lfloor \lg n \rfloor + 1$$

Conclusion: A binomial heap of size n has at most $\lfloor \lg n \rfloor + 1$ trees



Binomial Heaps – Structural Operations ($|H| = n$)

- **MakeHeap()**

 $\mathcal{O}(1)$

- Create an empty binomial heap (empty root list)



Binomial Heaps – Structural Operations ($|H| = n$)

- **MakeHeap()** $\mathcal{O}(1)$
 - Create an empty binomial heap (empty root list)
- **Minimum(H)** $\mathcal{O}(\lg n)$
 - Scan the roots of H to find the minimum-key root



Binomial Heaps – Structural Operations ($|H| = n$)

- **MakeHeap()** $\mathcal{O}(1)$
 - Create an empty binomial heap (empty root list)
- **Minimum(H)** $\mathcal{O}(\lg n)$
 - Scan the roots of H to find the minimum-key root
- **Link(y, x)** $\mathcal{O}(1)$



Binomial Heaps – Structural Operations ($|H| = n$)

- **MakeHeap()** $\mathcal{O}(1)$
 - Create an empty binomial heap (empty root list)
- **Minimum(H)** $\mathcal{O}(\lg n)$
 - Scan the roots of H to find the minimum-key root
- **Link(y, x)** $\mathcal{O}(1)$
 - Precondition: x and y are roots of two B_k trees
 - Make y a child of x (where $\text{key}[x] \leq \text{key}[y]$)
 - Result: one B_{k+1} tree



Binomial Heaps – Structural Operations ($|H| = n$)

- **MakeHeap()** $\mathcal{O}(1)$
 - Create an empty binomial heap (empty root list)
- **Minimum(H)** $\mathcal{O}(\lg n)$
 - Scan the roots of H to find the minimum-key root
- **Link(y, x)** $\mathcal{O}(1)$
- **Merge(H_1, H_2)** $\mathcal{O}(\lg n)$



Binomial Heaps – Structural Operations ($|H| = n$)

- **MakeHeap()** $\mathcal{O}(1)$
 - Create an empty binomial heap (empty root list)
- **Minimum(H)** $\mathcal{O}(\lg n)$
 - Scan the roots of H to find the minimum-key root
- **Link(y, x)** $\mathcal{O}(1)$
- **Merge(H_1, H_2)** $\mathcal{O}(\lg n)$
 - Merge the two root lists sorted by **degree**
 - Does *not* perform linking; duplicates of same degree may appear



Binomial Heaps – Structural Operations ($|H| = n$)

- **MakeHeap()** $\mathcal{O}(1)$
 - Create an empty binomial heap (empty root list)
- **Minimum(H)** $\mathcal{O}(\lg n)$
 - Scan the roots of H to find the minimum-key root
- **Link(y, x)** $\mathcal{O}(1)$
- **Merge(H_1, H_2)** $\mathcal{O}(\lg n)$
- **Union(H_1, H_2)** $\mathcal{O}(\lg n)$



Binomial Heaps – Structural Operations ($|H| = n$)

- **MakeHeap()** $\mathcal{O}(1)$
 - Create an empty binomial heap (empty root list)
- **Minimum(H)** $\mathcal{O}(\lg n)$
 - Scan the roots of H to find the minimum-key root
- **Link(y, x)** $\mathcal{O}(1)$
- **Merge(H_1, H_2)** $\mathcal{O}(\lg n)$
- **Union(H_1, H_2)** $\mathcal{O}(\lg n)$
 - **Step 1: Merge** the root lists of H_1 and H_2
 - **Step 2:** scan left-to-right, and when two adjacent roots have the same degree:
 - use **Link** to consolidate them
 - ensures at most one tree of each degree



Binomial Heaps – Priority Queue Operations

($|H| = n$)

- **Insert(H, x)**

worst $\mathcal{O}(\lg n)$, amort. $\mathcal{O}(1)$



Binomial Heaps – Priority Queue Operations

($|H| = n$)

- **Insert(H, x)** worst $\mathcal{O}(\lg n)$, amort. $\mathcal{O}(1)$
 - Construct a heap H_x containing the single node x (a B_0 tree)
 - Return **Union(H, H_x)**

$$(|H| = n)$$

- **Insert(H, x)** worst $\mathcal{O}(\lg n)$, amort. $\mathcal{O}(1)$
- **ExtractMin(H)** $\mathcal{O}(\lg n)$



Binomial Heaps – Priority Queue Operations

($|H| = n$)

- **Insert(H, x)** worst $\mathcal{O}(\lg n)$, amort. $\mathcal{O}(1)$
- **ExtractMin(H)** $\mathcal{O}(\lg n)$
 - Use **Minimum** to find the root r with minimum key
 - Remove r from the root list of H , and form a heap H_1 from the **children of** r (reverse their order so degrees increase)
 - Return **Union(H, H_1)** and the minimum element



Binomial Heaps – Priority Queue Operations

($|H| = n$)

- **Insert(H, x)** worst $\mathcal{O}(\lg n)$, amort. $\mathcal{O}(1)$
- **ExtractMin(H)** $\mathcal{O}(\lg n)$
- **DecreaseKey(H, x, newKey)** $\mathcal{O}(\lg n)$



Binomial Heaps – Priority Queue Operations

($|H| = n$)

- **Insert(H, x)** worst $\mathcal{O}(\lg n)$, amort. $\mathcal{O}(1)$
- **ExtractMin(H)** $\mathcal{O}(\lg n)$
- **DecreaseKey($H, x, newKey$)** $\mathcal{O}(\lg n)$
 - Set $key[x] \leftarrow newKey$
 - While heap-order is violated ($key[x] < key[parent(x)]$):
 - Exchange x with its parent (bubble-up)
 - Trees have height $\leq \lg n$, so at most $\mathcal{O}(\lg n)$ swaps



Binomial Heaps – Priority Queue Operations ($|H| = n$)

- **Insert(H, x)** worst $\mathcal{O}(\lg n)$, amort. $\mathcal{O}(1)$
- **ExtractMin(H)** $\mathcal{O}(\lg n)$
- **DecreaseKey(H, x, newKey)** $\mathcal{O}(\lg n)$
- **Delete(H, x)** $\mathcal{O}(\lg n)$



Binomial Heaps – Priority Queue Operations

$(|H| = n)$

- **Insert(H, x)** worst $\mathcal{O}(\lg n)$, amort. $\mathcal{O}(1)$
- **ExtractMin(H)** $\mathcal{O}(\lg n)$
- **DecreaseKey(H, x, newKey)** $\mathcal{O}(\lg n)$
- **Delete(H, x)** $\mathcal{O}(\lg n)$
 - Implemented by:
 - **DecreaseKey($H, x, -\infty$)**, then
 - **ExtractMin(H)**



From Binomial to Fibonacci Heaps

- Observation:
 - Binomial heaps maintain a **strict** structure: at most one B_k
 - Many operations do extra work just to keep this structure tight
- Fibonacci idea (high-level): **relax** the structure
 - relax the “one tree of each degree” invariant: consolidation only happens later, during ExtractMin
 - relax “strict degree” tree structure (child-loss tolerance): nodes are allowed to lose children without being immediately restructured (first child loss – mark the node; second child loss – cut node from parent (cascading))
- Result:
 - Several operations become $\mathcal{O}(1)$ *amortized*
 - We pay a larger (but still bounded !!) cost on ExtractMin

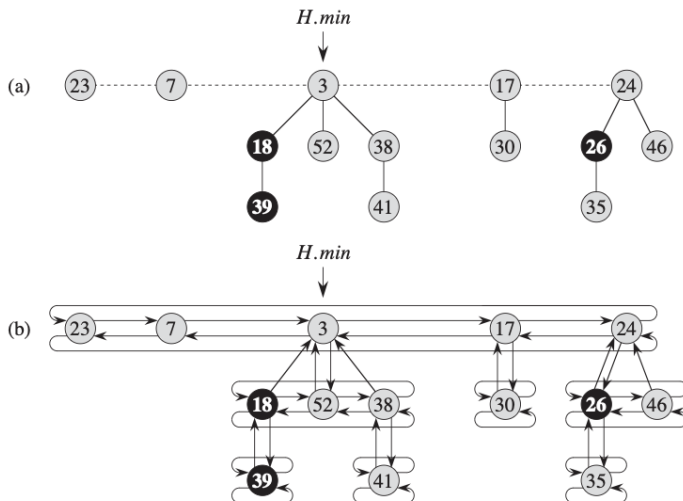


Fibonacci Heap: Structure

- A Fibonacci heap H is a collection of **heap-ordered trees** with:
 - A pointer $\text{min}[H]$ to the root with minimum key
 - Each node x has:
 - $\text{key}[x]$
 - $\text{degree}[x]$ = number of children
 - $\text{parent}[x]$, $\text{child}[x]$
 - $\text{left}[x]$, $\text{right}[x]$ – for circular, doubly-linked lists
 - $\text{mark}[x]$ – a boolean flag (used in decrease-key)
 - Roots form a **circular doubly-linked list**
 - Children of each node form another **circular doubly-linked list**
- Relaxed constraints:
 - May have several trees with the same degree
 - Tree shapes are not strictly binomial anymore



Fibonacci Heap Example





Fibonacci Heap: Simple Operations

- **Make-Heap():**



Fibonacci Heap: Simple Operations

- **Make-Heap():**
 - Create an empty heap (no trees, $\text{min}[H] = \text{NIL}$)



Fibonacci Heap: Simple Operations

- **Make-Heap():**
 - Create an empty heap (no trees, $\text{min}[H] = \text{NIL}$)
 - Time: $\mathcal{O}(1)$ worst-case



Fibonacci Heap: Simple Operations

- **Make-Heap():**
 - Create an empty heap (no trees, $\text{min}[H] = \text{NIL}$)
 - Time: $\mathcal{O}(1)$ worst-case
- **Insert(x):**



Fibonacci Heap: Simple Operations

- **Make-Heap():**
 - Create an empty heap (no trees, $\text{min}[H] = \text{NIL}$)
 - Time: $\mathcal{O}(1)$ worst-case
- **Insert(x):**
 - Make a 1-node tree



Fibonacci Heap: Simple Operations

- **Make-Heap():**
 - Create an empty heap (no trees, $\text{min}[H] = \text{NIL}$)
 - Time: $\mathcal{O}(1)$ worst-case
- **Insert(x):**
 - Make a 1-node tree
 - Add it to the **root list** of H



Fibonacci Heap: Simple Operations

- **Make-Heap():**
 - Create an empty heap (no trees, $\text{min}[H] = \text{NIL}$)
 - Time: $\mathcal{O}(1)$ worst-case
- **Insert(x):**
 - Make a 1-node tree
 - Add it to the **root list** of H
 - Update $\text{min}[H]$ if needed



Fibonacci Heap: Simple Operations

- **Make-Heap():**
 - Create an empty heap (no trees, $\text{min}[H] = \text{NIL}$)
 - Time: $\mathcal{O}(1)$ worst-case
- **Insert(x):**
 - Make a 1-node tree
 - Add it to the **root list** of H
 - Update $\text{min}[H]$ if needed
 - Time: $\mathcal{O}(1)$ worst-case



Fibonacci Heap: Simple Operations

- **Make-Heap():**
 - Create an empty heap (no trees, $\text{min}[H] = \text{NIL}$)
 - Time: $\mathcal{O}(1)$ worst-case
- **Insert(x):**
 - Make a 1-node tree
 - Add it to the **root list** of H
 - Update $\text{min}[H]$ if needed
 - Time: $\mathcal{O}(1)$ worst-case
- **FindMin():**



Fibonacci Heap: Simple Operations

- **Make-Heap():**
 - Create an empty heap (no trees, $\text{min}[H] = \text{NIL}$)
 - Time: $\mathcal{O}(1)$ worst-case
- **Insert(x):**
 - Make a 1-node tree
 - Add it to the **root list** of H
 - Update $\text{min}[H]$ if needed
 - Time: $\mathcal{O}(1)$ worst-case
- **FindMin():**
 - Just return $\text{min}[H]$



Fibonacci Heap: Simple Operations

- **Make-Heap():**
 - Create an empty heap (no trees, $\text{min}[H] = \text{NIL}$)
 - Time: $\mathcal{O}(1)$ worst-case
- **Insert(x):**
 - Make a 1-node tree
 - Add it to the **root list** of H
 - Update $\text{min}[H]$ if needed
 - Time: $\mathcal{O}(1)$ worst-case
- **FindMin():**
 - Just return $\text{min}[H]$
 - Time: $\mathcal{O}(1)$ worst-case



Fibonacci Heap: Simple Operations

- **Make-Heap():**
 - Create an empty heap (no trees, $\text{min}[H] = \text{NIL}$)
 - Time: $\mathcal{O}(1)$ worst-case
- **Insert(x):**
 - Make a 1-node tree
 - Add it to the **root list** of H
 - Update $\text{min}[H]$ if needed
 - Time: $\mathcal{O}(1)$ worst-case
- **FindMin():**
 - Just return $\text{min}[H]$
 - Time: $\mathcal{O}(1)$ worst-case
- **Union(H1, H2):**



Fibonacci Heap: Simple Operations

- **Make-Heap():**
 - Create an empty heap (no trees, $\text{min}[H] = \text{NIL}$)
 - Time: $\mathcal{O}(1)$ worst-case
- **Insert(x):**
 - Make a 1-node tree
 - Add it to the **root list** of H
 - Update $\text{min}[H]$ if needed
 - Time: $\mathcal{O}(1)$ worst-case
- **FindMin():**
 - Just return $\text{min}[H]$
 - Time: $\mathcal{O}(1)$ worst-case
- **Union(H1, H2):**
 - Concatenate the two root lists



Fibonacci Heap: Simple Operations

- **Make-Heap():**
 - Create an empty heap (no trees, $\text{min}[H] = \text{NIL}$)
 - Time: $\mathcal{O}(1)$ worst-case
- **Insert(x):**
 - Make a 1-node tree
 - Add it to the **root list** of H
 - Update $\text{min}[H]$ if needed
 - Time: $\mathcal{O}(1)$ worst-case
- **FindMin():**
 - Just return $\text{min}[H]$
 - Time: $\mathcal{O}(1)$ worst-case
- **Union(H1, H2):**
 - Concatenate the two root lists
 - Set $\text{min}[H]$ as min of the two minima



Fibonacci Heap: Simple Operations

- **Make-Heap():**
 - Create an empty heap (no trees, $\text{min}[H] = \text{NIL}$)
 - Time: $\mathcal{O}(1)$ worst-case
- **Insert(x):**
 - Make a 1-node tree
 - Add it to the **root list** of H
 - Update $\text{min}[H]$ if needed
 - Time: $\mathcal{O}(1)$ worst-case
- **FindMin():**
 - Just return $\text{min}[H]$
 - Time: $\mathcal{O}(1)$ worst-case
- **Union(H1, H2):**
 - Concatenate the two root lists
 - Set $\text{min}[H]$ as min of the two minima
 - Time: $\mathcal{O}(1)$ worst-case



Fibonacci Heap: ExtractMin (1/2)

- **ExtractMin(H):**
 - 1 Let $z = \min[H]$
 - 2 For each child x of z :
 - Add x to the root list of H
 - Set $\text{parent}[x] = \text{NIL}$
 - 3 Remove z from the root list
- If z was the only node:
 - Heap becomes empty
- Otherwise:
 - Choose an arbitrary root as temporary $\min[H]$
 - Call a **consolidation** procedure



Fibonacci Heap: ExtractMin (2/2) – Consolidation

Goal: After removing $z = \min[H]$, ensure there is **at most one root of each degree** (as in binomial heaps) & reestablish new $\min[H]$

1. **Initialize the degree table A :** $A[0..D] \leftarrow \text{NIL}$, where $D = O(\log n)$



Fibonacci Heap: ExtractMin (2/2) – Consolidation

Goal: After removing $z = \min[H]$, ensure there is **at most one root of each degree** (as in binomial heaps) & reestablish new $\min[H]$

1. **Initialize the degree table** A : $A[0..D] \leftarrow \text{NIL}$, where $D = O(\log n)$
2. **Traverse the root list**

- **for** each root x encountered (having crt degree d):
 - **while** $A[d] \neq \text{NIL}$:
 - Let $y = A[d]$ be the previous root of degree d (encountered earlier during traversal)
 - **Compare keys of the two roots** x and y
 - **Link** the larger-key root under the smaller-key root
 - $A[d] = \text{NIL}$; $d = d + 1$ // *potentially cascade*
 - Set x to the resulting (linked) root
 - Set $A[d] = x$
- Note: we **do not** scan A ; access $A[d]$ acc. to the degrees found



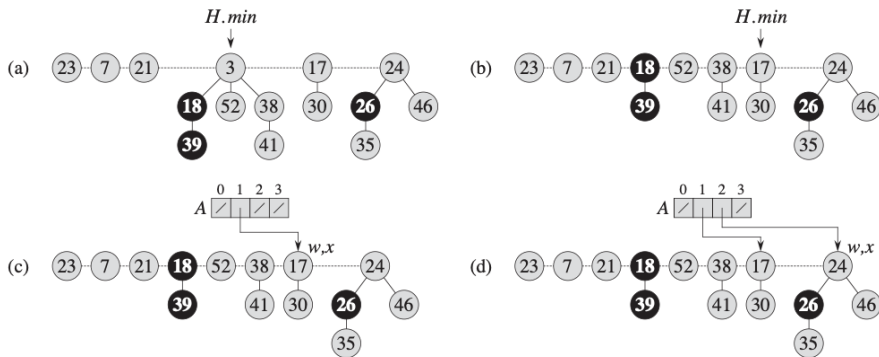
Fibonacci Heap: ExtractMin (2/2) – Consolidation

Goal: After removing $z = \min[H]$, ensure there is **at most one root of each degree** (as in binomial heaps) & reestablish new $\min[H]$

1. **Initialize the degree table A :** $A[0..D] \leftarrow \text{NIL}$, where $D = O(\log n)$
2. **Traverse the root list and ensure at most one tree of each degree in $A[0..D]$**
3. **Rebuild the root list**
 - **traverse $A[0..D]$**
 - **collect all non-NIL entries as the new root list**
 - **update $\min[H]$**

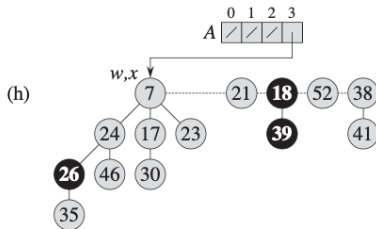
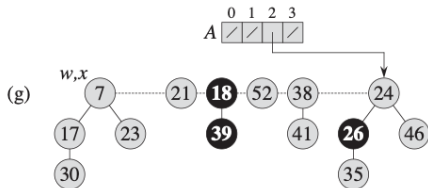
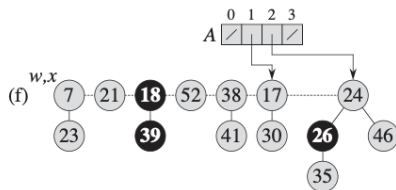
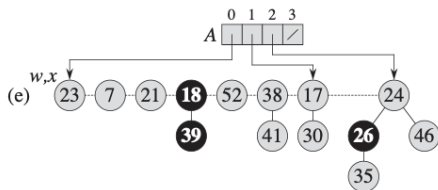


Fibonacci Heap: ExtractMin



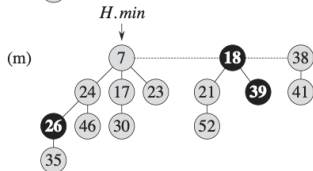
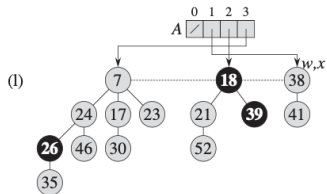
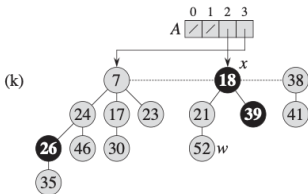
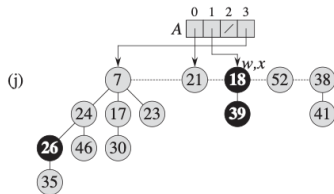
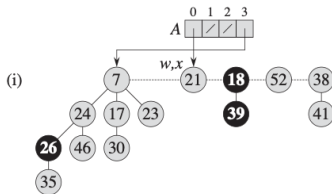


Fibonacci Heap: ExtractMin





Fibonacci Heap: ExtractMin





Fibonacci Heap: DecreaseKey – Cascading Cuts

Goal: decrease the key of node x to $k < \text{key}[x]$ and restore heap

1. Update key

$\text{key}[x] \leftarrow k$

if x is a root or $\text{key}[\text{parent}[x]] \leq \text{key}[x]$, stop

2. CUT($x, \text{parent}[x]$)

remove x from its parent's child list

add x to the root list of H

clear $\text{mark}[x]$

3. CASCADING-CUT($\text{parent}[x]$)

let $y = \text{parent}[x]$

if y is **unmarked**: mark it (first child loss)

if y is **marked**:

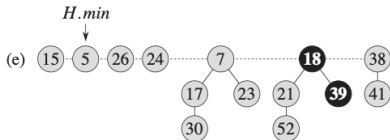
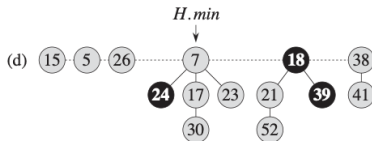
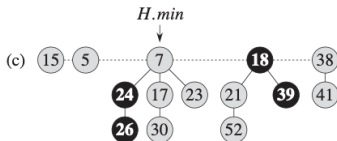
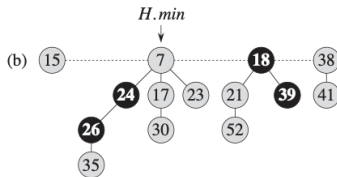
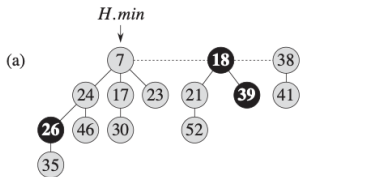
CUT($y, \text{parent}[y]$)

CASCADING-CUT($\text{parent}[y]$)

4. Update $\text{min}[H]$, if necessary



Fibonacci Heap: DecreaseKey ($46 \leftarrow 15$ (a-b); $35 \leftarrow 5$ (c-e))





Fibonacci Heaps – Potential Function

To analyze amortized complexity, we use a potential function:

$$\Phi(H) = t(H) + 2m(H),$$

- $t(H)$ = number of **trees** (roots) in the root list
- $m(H)$ = number of **marked nodes**

Intuition: future work encoded as potential

- More trees $\Rightarrow$ more work in the next **consolidation**



Fibonacci Heaps – Potential Function

To analyze amortized complexity, we use a potential function:

$$\Phi(H) = t(H) + 2m(H),$$

- $t(H)$ = number of **trees** (roots) in the root list
- $m(H)$ = number of **marked nodes**

Intuition: future work encoded as potential

- More trees $\Rightarrow$ more work in the next **consolidation**
- More marked nodes $\Rightarrow$ more cascading **cuts** may be needed



Fibonacci Heaps – Potential Function

To analyze amortized complexity, we use a potential function:

$$\Phi(H) = t(H) + 2m(H),$$

- $t(H)$ = number of **trees** (roots) in the root list
- $m(H)$ = number of **marked nodes**

Intuition: future work encoded as potential

- More trees $\Rightarrow$ more work in the next **consolidation**
- More marked nodes $\Rightarrow$ more cascading **cuts** may be needed
- $\Phi(H)$ measures the “disorder” left by lazy operations
(coefficients are chosen so that links and cascading cuts both cause a drop of at least 1 in potential, exactly covering their constant actual cost)



Fibonacci Heaps – Potential Function

To analyze amortized complexity, we use a potential function:

$$\Phi(H) = t(H) + 2m(H),$$

- $t(H)$ = number of **trees** (roots) in the root list
- $m(H)$ = number of **marked nodes**

Intuition: future work encoded as potential

- More trees $\Rightarrow$ more work in the next **consolidation**
- More marked nodes $\Rightarrow$ more cascading **cuts** may be needed
- $\Phi(H)$ measures the “disorder” left by lazy operations (coefficients are chosen so that links and cascading cuts both cause a drop of at least 1 in potential, exactly covering their constant actual cost)

Amortized cost of an operation:

$$\hat{c} = c_{\text{actual}} + (\Phi_{\text{after}} - \Phi_{\text{before}})$$



Fibonacci Heaps – Amortized Cost of ExtractMin

Actual work:

- Remove min root z
- Add its $d(z)$ children to the root list
- Consolidate: traverse root list, link eq.-degree trees, rebuild list
- Worst-case actual cost: $\Theta(t(H) + D(N))$, where $t(H)$ is #roots before consolidation, $D(N)$ is the max. degree of any node in H



Fibonacci Heaps – Amortized Cost of ExtractMin

Actual work:

- Remove min root z
- Add its $d(z)$ children to the root list
- Consolidate: traverse root list, link eq.-degree trees, rebuild list
- Worst-case actual cost: $\Theta(t(H) + D(N))$, where $t(H)$ is #roots before consolidation, $D(N)$ is the max. degree of any node in H
- The potential before extracting the minimum node:

$$\Phi_{\text{before}} = t(H) + 2 m(H)$$



Fibonacci Heaps – Amortized Cost of ExtractMin

Actual work:

- Remove min root z
- Add its $d(z)$ children to the root list
- Consolidate: traverse root list, link eq.-degree trees, rebuild list
- Worst-case actual cost: $\Theta(t(H) + D(N))$, where $t(H)$ is #roots before consolidation, $D(N)$ is the max. degree of any node in H
- The potential before extracting the minimum node:

$$\Phi_{\text{before}} = t(H) + 2 m(H)$$

- The potential after the consolidation:

$$\Phi_{\text{after}} \leq (D(n) + 1) + 2 m(H)$$



Fibonacci Heaps – Amortized Cost of ExtractMin

Actual work:

- Remove min root z
- Add its $d(z)$ children to the root list
- Consolidate: traverse root list, link eq.-degree trees, rebuild list
- Worst-case actual cost: $\Theta(t(H) + D(N))$, where $t(H)$ is #roots before consolidation, $D(N)$ is the max. degree of any node in H
- The potential before extracting the minimum node:

$$\Phi_{\text{before}} = t(H) + 2 m(H)$$

- The potential after the consolidation:

$$\Phi_{\text{after}} \leq (D(n) + 1) + 2 m(H)$$

- Then: $\hat{c} = c_{\text{actual}} + (\Phi_{\text{after}} - \Phi_{\text{before}}) \leq$
 $\mathcal{O}(D(n) + t(H)) + ((D(n) + 1 + 2m(H)) - (t(H) + 2m(H)))$



Fibonacci Heaps – Amortized Cost of ExtractMin

Actual work:

- Remove min root z
- Add its $d(z)$ children to the root list
- Consolidate: traverse root list, link eq.-degree trees, rebuild list
- Worst-case actual cost: $\Theta(t(H) + D(N))$, where $t(H)$ is #roots before consolidation, $D(N)$ is the max. degree of any node in H
- Then: $\hat{c} = c_{\text{actual}} + (\Phi_{\text{after}} - \Phi_{\text{before}}) \leq$
 $\mathcal{O}(D(n) + t(H)) + ((D(n) + 1 + 2m(H)) - (t(H) + 2m(H)))$
 $= \mathcal{O}(D(n)) + \mathcal{O}(t(H)) - t(H)$
 $= \mathcal{O}(D(n))$



Fibonacci Heaps – Amortized Cost of ExtractMin

Actual work:

- Remove min root z
- Add its $d(z)$ children to the root list
- Consolidate: traverse root list, link eq.-degree trees, rebuild list
- Worst-case actual cost: $\Theta(t(H) + D(N))$, where $t(H)$ is #roots before consolidation, $D(N)$ is the max. degree of any node in H
- Then: $\hat{c} = c_{\text{actual}} + (\Phi_{\text{after}} - \Phi_{\text{before}}) \leq$
 $\mathcal{O}(D(n) + t(H)) + ((D(n) + 1 + 2m(H)) - (t(H) + 2m(H)))$
 $= \mathcal{O}(D(n)) + \mathcal{O}(t(H)) - t(H)$
 $= \mathcal{O}(D(n))$

We can scale the potential so that the $-t(H)$ term dominates the constant hidden in $\mathcal{O}(t(H))$



Fibonacci Heaps – Amortized Cost of ExtractMin

Actual work:

- Remove min root z
- Add its $d(z)$ children to the root list
- Consolidate: traverse root list, link eq.-degree trees, rebuild list
- Worst-case actual cost: $\Theta(t(H) + D(N))$, where $t(H)$ is #roots before consolidation, $D(N)$ is the max. degree of any node in H
- Then: $\hat{c} = c_{\text{actual}} + (\Phi_{\text{after}} - \Phi_{\text{before}}) \leq$
 $\mathcal{O}(D(n) + t(H)) + ((D(n) + 1 + 2m(H)) - (t(H) + 2m(H)))$
 $= \mathcal{O}(D(n)) + \mathcal{O}(t(H)) - t(H)$
 $= \mathcal{O}(D(n))$

We can scale the potential so that the $-t(H)$ term dominates the constant hidden in $\mathcal{O}(t(H))$

- Intuitively: each link reduces the number of roots by 1, producing a drop in potential that pays for the cost of the link



Fibonacci Heaps – DecreaseKey: Structure

Steps:

- Decrease $\text{key}[x]$
- If heap order holds, stop
- Otherwise:
 - $\text{CUT}(x, \text{parent}[x])$,
 - CASCADING-CUT on ancestors

Let c be the total number of cuts (1 cut of x + cascading cuts)

Actual cost:

$$c_{\text{actual}} = \mathcal{O}(c + 1)$$

Effect of a single cut:

- Adds a new tree: $t = t + 1$
- Clears a mark: $m = m - 1$ (if node was marked)



Fibonacci Heaps – Amortized Cost of DecreaseKey

In the full operation (1 cut of $x + c - 1$ cascading cuts):

Changes to potential:

- Trees: $\Delta t = +c$
- Marks:
 - At most one new mark: $+1$,
 - At least $(c - 1)$ marks removed: $-(c - 1)$
- So:

$$\Delta m \leq 1 - (c - 1) = 2 - c$$

- Potential change:

$$\Delta \Phi = \Delta t + 2\Delta m \leq c + 2(2 - c) = 4 - c$$

Amortized cost:

$$\hat{c} = c_{\text{actual}} + \Delta \Phi \leq \mathcal{O}(c + 1) + (4 - c) = \mathcal{O}(1)$$



Why Rank-Pairing Heaps?

- Fibonacci heaps are **theoretically beautiful**, but:
 - Implementation is complex: many pointers, circular lists, marks, cascading cuts
 - Harder to debug and maintain
- Goal:
 - Keep the **same asymptotic bounds** as Fibonacci heaps
 - Use a **simpler structure** and simpler decrease-key logic
- One such structure: **Rank-pairing heaps** (Tarjan & Raman, 1997)
 - Designed to have:
 - **Amortized** Insert/DecreaseKey/FindMin/Union in $\mathcal{O}(1)$
 - **Amortized** ExtractMin in $\mathcal{O}(\log n)$
 - But with fewer structural invariants than Fibonacci heaps



Rank-Pairing Heap – High-Level Structure

- Still a collection of **heap-ordered trees**
- Each node x has a **rank** $r(x)$:
 - An integer that places an upper bound on the height (size of each tree is exponential in rank)
 - Similar spirit to *degree* in Fibonacci heaps, but used differently
 - Rank approximates the "binomial-like" structure without maintaining as strict a shape
- Children are linked in a simple way (e. g., singly-linked lists or simple sibling pointers)
- Key difference:
 - avoid explicit **mark bits** and complicated cascading cut rules
 - instead, impose simple constraints on how ranks of parent/children relate



Rank-Pairing Heap – Operations (Intuition)

- **Make-Heap, Insert, FindMin, Union:**

- Very similar to Fibonacci heaps:
 - Insert: create 1-node tree and add to root list
 - Union: concatenate root lists and adjust min
- Amortized $\mathcal{O}(1)$

- **ExtractMin:**

- Remove the root with minimum key
- Add its children to the root list
- Perform a consolidation step guided by ranks:
 - Trees with the same rank are linked (like binomial/Fibonacci)
- Amortized $\mathcal{O}(\log n)$



Rank-Pairing Heap – DecreaseKey (Intuition)

- **DecreaseKey(H, x, newKey):**
 - Decrease $\text{key}[x]$ to newKey
 - If heap property is still valid, stop
 - Otherwise:
 - Cut x from its parent and add to root list (like in Fibonacci)
 - But instead of a full cascading-cut mechanism:
 - Adjust some node ranks in a simple way
 - Possibly perform a small number of additional cuts
- **Analysis:**
 - Rank rules ensure that:
 - The rank of a node is bounded by $\mathcal{O}(\log n)$
 - The number of cuts per decreaseKey is small *on average*
 - $\Rightarrow$ **amortized** $\mathcal{O}(1)$



Binomial vs Fibonacci vs Rank-Pairing

Operation	Binomial Heap	Fibonacci Heap	Rank-Pairing Heap
Make-Heap	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
FindMin	$\mathcal{O}(\log n)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Insert	$\mathcal{O}(\log n)$ (worst)	$\mathcal{O}(1)$ (amort.)	$\mathcal{O}(1)$ (amort.)
Union	$\mathcal{O}(\log n)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
ExtractMin	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$ (amort.)	$\mathcal{O}(\log n)$ (amort.)
DecreaseKey	$\mathcal{O}(\log n)$	$\mathcal{O}(1)$ (amort.)	$\mathcal{O}(1)$ (amort.)

Binomial heaps: conceptually clean, good “bridge” from binary heaps

Fibonacci heaps: best-known asymptotic bounds, conceptually rich but complex

Rank-pairing heaps: similar amortized bounds, simpler structure, closer to practice



Which Priority Queue Should I Use?

- **Binary heap:**

Easiest to implement, very good constants

Usually the best choice in practice, unless you do *a lot* of DecreaseKey ops

- **Binomial heap:**

Good for teaching and reasoning about **mergeable heaps**

Useful when **Merge** is very frequent

- **Fibonacci / Rank-pairing heaps:**

Shine in **theoretical** analysis (e. g., asymptotically optimal implementations of Dijkstra, Prim)

Rank-pairing heaps: better balance between theory and implementability

In real code:

Often a binary heap or pairing heap is used

But understanding Fibonacci-style heaps gives deep insight into:

- Amortized analysis

- Laziness and structural relaxation

- Trade-off between invariants and performance



Bibliography

- Cormen, Thomas H., et al., *"Introduction to algorithms."*, MIT press, 2009, chapter 19
- Supplementary (not mandatory!): Rank-Pairing Heaps Paper